# Project: Student Academic Performance & Analytics Management System

## 1. Project Objective

Build a complete Python-based system that manages student academic data and produces analytical insights.

The system should:

- Store student information
- Store subject-wise marks
- Calculate GPA, percentage and grades
- Analyse attendance
- Identify academically weak students
- Generate department-wise statistics
- Compare semester performance
- Rank students
- Identify at-risk students
- Perform statistical analysis using NumPy
- Perform data manipulation using Pandas
- Generate reports
- Export results to CSV/Excel
- Use **classes and object-oriented programming throughout**

# 2. Technologies

```
Python
│
├── OOP
│    ├── Classes
│    ├── Objects
│    ├── Inheritance
│    ├── Encapsulation
│    ├── Polymorphism
│    └── Class methods
│
├── NumPy
│    ├── Arrays
│    ├── Statistics
│    ├── Matrix operations
│    └── Mathematical calculations
│
├── Pandas
│    ├── Series
│    ├── DataFrame
│    ├── Filtering
│    ├── GroupBy
│    ├── Merge
│    ├── Aggregation
│    └── CSV/Excel
│
└── Matplotlib / Seaborn
```

```
    └── Data visualization
```

# 3. Suggested Dataset

Create a dataset containing approximately **500 students**.

Each student can have:

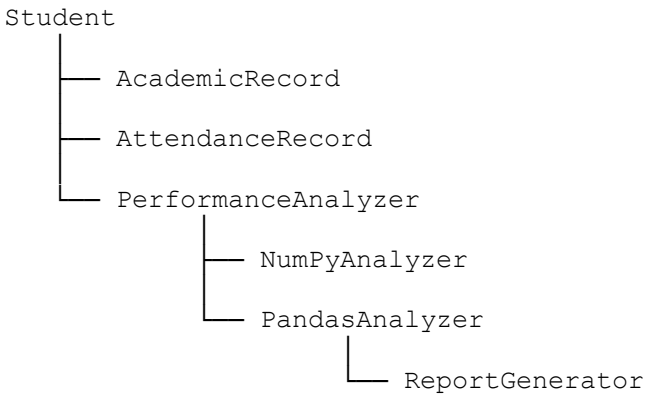
```
Student_ID
Name
Gender
Department
Year
Semester
Age
Attendance
Python
Statistics
Machine_Learning
Deep_Learning
Database
Data_Visualization
Assignment_Score
Internal_Marks
Project_Marks
Study_Hours
Placement_Status
```

Example:

```
students = [
    {
        "Student_ID": "DS001",
        "Name": "Arun Kumar",
        "Gender": "Male",
        "Department": "Data Science",
        "Year": 1,
        "Semester": 2,
        "Age": 22,
        "Attendance": 87,
        "Python": 78,
        "Statistics": 82,
        "Machine_Learning": 75,
        "Deep_Learning": 69,
        "Database": 88,
        "Data_Visualization": 91,
        "Assignment_Score": 85,
        "Internal_Marks": 82,
        "Project_Marks": 88,
        "Study_Hours": 18,
        "Placement_Status": "Not Placed"
    }
]
```

# 4. Class Architecture

The project should have several classes rather than putting everything inside one large program.




A possible class structure:

```
class Student:
    pass


class AcademicRecord:
    pass


class AttendanceRecord:
    pass


class StudentDatabase:
    pass


class PerformanceAnalyzer:
    pass


class NumPyAnalyzer:
    pass


class PandasAnalyzer:
    pass


class ReportGenerator:
    pass
```

# 5. Student Class

Create a `Student` class.

```
class Student:

    def __init__(self, student_id, name, gender,
```

```
                 department, year, semester, age):

        self.student_id = student_id
        self.name = name
        self.gender = gender
        self.department = department
        self.year = year
        self.semester = semester
        self.age = age

    def display(self):

        print("Student ID :", self.student_id)
        print("Name       :", self.name)
        print("Department :", self.department)
        print("Year       :", self.year)
        print("Semester   :", self.semester)
```

Create objects:

```
s1 = Student(
    "DS001",
    "Arun Kumar",
    "Male",
    "Data Science",
    1,
    2,
    22
)

s1.display()
```

# 6. AcademicRecord Class

```
class AcademicRecord:

    def __init__(self, student_id, marks):

        self.student_id = student_id
        self.marks = marks

    def total(self):

        return sum(self.marks.values())

    def average(self):

        return self.total() / len(self.marks)

    def highest(self):

        return max(self.marks.values())

    def lowest(self):

        return min(self.marks.values())

    def grade(self):
```

```
        avg = self.average()

        if avg >= 90:
            return "A+"
        elif avg >= 80:
            return "A"
        elif avg >= 70:
            return "B"
        elif avg >= 60:
            return "C"
        elif avg >= 50:
            return "D"
        else:
            return "F"
```

# 7. Using NumPy

Instead of Python's normal `sum()`, introduce NumPy.

```
import numpy as np

marks = np.array([
    78, 82, 75, 69, 88, 91
])

print("Mean:", np.mean(marks))
print("Median:", np.median(marks))
print("Maximum:", np.max(marks))
print("Minimum:", np.min(marks))
print("Standard deviation:", np.std(marks))
print("Variance:", np.var(marks))
```

Students should understand why NumPy is useful for numerical data.

# 8. NumPyAnalyzer Class

```
class NumPyAnalyzer:

    def __init__(self, marks):

        self.marks = np.array(marks)

    def mean(self):

        return np.mean(self.marks)

    def median(self):

        return np.median(self.marks)

    def standard_deviation(self):

        return np.std(self.marks)

    def variance(self):
```

```
        return np.var(self.marks)

    def percentile(self, value):

        return np.percentile(self.marks, value)

    def statistics(self):

        return {
            "Mean": self.mean(),
            "Median": self.median(),
            "Standard Deviation": self.standard_deviation(),
            "Variance": self.variance(),
            "25th Percentile": self.percentile(25),
            "75th Percentile": self.percentile(75)
        }
```

# 9. Pandas DataFrame

Convert all student records into a DataFrame.

```
import pandas as pd

df = pd.DataFrame(students)

print(df.head())
print(df.shape)
print(df.columns)
print(df.info())
```

Students should learn:

```
df.head()
df.tail()
df.shape
df.info()
df.describe()
df.dtypes
```

# 10. Data Cleaning

Introduce missing values deliberately.

```
df.loc[5, "Python"] = np.nan
df.loc[12, "Attendance"] = np.nan
df.loc[20, "Study_Hours"] = np.nan
```

Find missing values:

```
print(df.isnull().sum())
```

Fill numerical missing values:

```
df["Python"] = df["Python"].fillna(
```

```
    df["Python"].mean()
)

df["Attendance"] = df["Attendance"].fillna(
    df["Attendance"].mean()
)
```

# 11. Feature Engineering

Create a total marks column.

```
subjects = [
    "Python",
    "Statistics",
    "Machine_Learning",
    "Deep_Learning",
    "Database",
    "Data_Visualization"
]

df["Total_Marks"] = df[subjects].sum(axis=1)
```

Calculate percentage:

```
df["Percentage"] = (
    df["Total_Marks"] /
    (len(subjects) * 100)
) * 100
```

# 12. Grade Classification

```
def calculate_grade(mark):

    if mark >= 90:
        return "A+"
    elif mark >= 80:
        return "A"
    elif mark >= 70:
        return "B"
    elif mark >= 60:
        return "C"
    elif mark >= 50:
        return "D"
    else:
        return "F"
```

Apply the function:

```
df["Grade"] = df["Percentage"].apply(
    calculate_grade
)
```

# 13. Ranking Students

```
df["Rank"] = (
    df["Percentage"]
    .rank(
        ascending=False,
        method="dense"
    )
)
```

Display top 10:

```
top_students = df.sort_values(
    "Percentage",
    ascending=False
).head(10)

print(top_students[
    ["Student_ID", "Name", "Percentage", "Rank"]
])
```

# 14. Department Analysis

```
department_analysis = df.groupby(
    "Department"
)["Percentage"].agg([
    "mean",
    "max",
    "min",
    "std"
])

print(department_analysis)
```

This gives students practical experience with:

```
groupby()
agg()
mean()
max()
min()
std()
```

# 15. Gender Analysis

```
gender_analysis = df.groupby(
    "Gender"
)["Percentage"].mean()

print(gender_analysis)
```

More advanced:

```
gender_department = pd.pivot_table(
    df,
    values="Percentage",
    index="Department",
```

```
    columns="Gender",
    aggfunc="mean"
)

print(gender_department)
```

# 16. Attendance Analysis

Create attendance categories.

```
def attendance_status(attendance):

    if attendance >= 85:
        return "Excellent"

    elif attendance >= 75:
        return "Good"

    elif attendance >= 65:
        return "Warning"

    else:
        return "Critical"
df["Attendance_Status"] = (
    df["Attendance"]
    .apply(attendance_status)
)
```

Find students below 75%.

```
low_attendance = df[
    df["Attendance"] < 75
]

print(low_attendance)
```

# 17. At-Risk Student Identification

This is where the project becomes much more interesting.

A student is considered **at risk** if:

```
Attendance < 75
OR
Percentage < 50
OR
Study hours < 10
OR
failed in one or more subjects
df["At_Risk"] = (
    (df["Attendance"] < 75) |
    (df["Percentage"] < 50) |
    (df["Study_Hours"] < 10)
)
```

Then:

```
at_risk_students = df[
    df["At_Risk"] == True
]

print(at_risk_students)
```

# 18. Advanced Risk Score

Create a numerical risk score.

```
df["Risk_Score"] = 0

df.loc[
    df["Attendance"] < 75,
    "Risk_Score"
] += 1

df.loc[
    df["Percentage"] < 50,
    "Risk_Score"
] += 2

df.loc[
    df["Study_Hours"] < 10,
    "Risk_Score"
] += 1
```

Classify:

```
def risk_category(score):

    if score >= 3:
        return "High Risk"

    elif score == 2:
        return "Medium Risk"

    elif score == 1:
        return "Low Risk"

    return "Safe"
df["Risk_Category"] = (
    df["Risk_Score"]
    .apply(risk_category)
)
```

# 19. Correlation Analysis Using NumPy

Study the relationship between:

```
Study Hours
Attendance
Internal Marks
```

```
Percentage
variables = df[
    [
        "Study_Hours",
        "Attendance",
        "Internal_Marks",
        "Percentage"
    ]
]

correlation = np.corrcoef(
    variables.T
)

print(correlation)
```

Students can interpret:

```
+1  → Strong positive relationship
 0  → No linear relationship
-1  → Strong negative relationship
```

# 20. Pandas Correlation

```
correlation_matrix = df[
    [
        "Study_Hours",
        "Attendance",
        "Internal_Marks",
        "Percentage"
    ]
].corr()

print(correlation_matrix)
```

Compare NumPy and Pandas results.

# 21. Subject-Wise Analysis

```
subject_average = df[subjects].mean()

print(subject_average)
```

Find the weakest subject:

```
weakest_subject = subject_average.idxmin()

print(
    "Weakest Subject:",
    weakest_subject
)
```

Find the strongest:

```
strongest_subject = subject_average.idxmax()
```

```
print(
    "Strongest Subject:",
    strongest_subject
)
```

# 22. Pass Percentage

```
pass_percentage = (
    (df["Percentage"] >= 50).mean()
) * 100

print(
    "Overall Pass Percentage:",
    pass_percentage
)
```

Department-wise:

```
pass_by_department = (
    df.groupby("Department")
      ["Percentage"]
      .apply(lambda x: (x >= 50).mean() * 100)
)

print(pass_by_department)
```

# 23. Visualization

Introduce Matplotlib.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

plt.hist(
    df["Percentage"],
    bins=10
)

plt.xlabel("Percentage")
plt.ylabel("Number of Students")
plt.title("Distribution of Student Performance")

plt.show()
```

Department comparison:

```
department_avg = (
    df.groupby("Department")
      ["Percentage"]
      .mean()
)

department_avg.plot(
```

```
    kind="bar",
    figsize=(10, 6)
)

plt.title("Average Performance by Department")
plt.ylabel("Average Percentage")

plt.show()
```

# 24. Object-Oriented Data Analyzer

Create a central analyzer class.

```
class PerformanceAnalyzer:

    def __init__(self, dataframe):

        self.df = dataframe

    def average_percentage(self):

        return self.df["Percentage"].mean()

    def highest_percentage(self):

        return self.df["Percentage"].max()

    def lowest_percentage(self):

        return self.df["Percentage"].min()

    def topper(self):

        index = self.df["Percentage"].idxmax()

        return self.df.loc[index]

    def department_average(self):

        return (
            self.df
            .groupby("Department")
            ["Percentage"]
            .mean()
        )

    def attendance_average(self):

        return self.df["Attendance"].mean()

    def at_risk_students(self):

        return self.df[
            self.df["At_Risk"] == True
        ]
```

# 25. Report Generator

```
class ReportGenerator:

    def __init__(self, dataframe):

        self.df = dataframe

    def student_report(self, student_id):

        result = self.df[
            self.df["Student_ID"] == student_id
        ]

        if result.empty:

            print("Student not found.")

        else:

            print(result.T)

    def top_students(self, n=10):

        return (
            self.df
            .sort_values(
                "Percentage",
                ascending=False
            )
            .head(n)
        )

    def export_csv(self, filename):

        self.df.to_csv(
            filename,
            index=False
        )
```

# 26. Main Application

Finally, students should create a menu-driven application.

```
def main():

    analyzer = PerformanceAnalyzer(df)

    report = ReportGenerator(df)

    while True:

        print("\n==============================")
        print(" STUDENT ANALYTICS SYSTEM")
        print("==============================")

        print("1. Display Students")
        print("2. Show Top Students")
        print("3. Department Analysis")
        print("4. Subject Analysis")
        print("5. Attendance Analysis")
```

```
print("6. At-Risk Students")
print("7. Student Search")
print("8. Overall Statistics")
print("9. Export Report")
print("10. Exit")

choice = input(
    "Enter your choice: "
)

if choice == "1":

    print(df.head(20))

elif choice == "2":

    print(report.top_students())

elif choice == "3":

    print(
        analyzer.department_average()
    )

elif choice == "4":

    print(
        df[subjects].mean()
    )

elif choice == "5":

    print(
        df["Attendance"]
        .describe()
    )

elif choice == "6":

    print(
        analyzer.at_risk_students()
    )

elif choice == "7":

    sid = input(
        "Enter Student ID: "
    )

    report.student_report(sid)

elif choice == "8":

    print(
        "Average:",
        analyzer.average_percentage()
    )

    print(
        "Highest:",
        analyzer.highest_percentage()
```

```
            )

            print(
                "Lowest:",
                analyzer.lowest_percentage()
            )

        elif choice == "9":

            report.export_csv(
                "student_report.csv"
            )

            print(
                "Report exported successfully."
            )

        elif choice == "10":

            print("Thank you!")

            break

        else:

            print("Invalid choice.")
```

# 27. Make It a Really Large PG-Level Project

For an **MSc Data Science project**, I would extend it into the following modules:

```
Student Analytics System
│
├── 1. Student Management
│       ├── Add student
│       ├── Update student
│       ├── Delete student
│       └── Search student
│
├── 2. Academic Management
│       ├── Marks
│       ├── Grades
│       ├── GPA
│       └── Semester performance
│
├── 3. Attendance Management
│       ├── Attendance percentage
│       ├── Shortage detection
│       └── Attendance trends
│
├── 4. NumPy Analytics
│       ├── Mean
│       ├── Median
│       ├── Variance
│       ├── Standard deviation
│       ├── Percentiles
```

```
│      └── Correlation
│
├── 5. Pandas Analytics
│      ├── Filtering
│      ├── Sorting
│      ├── GroupBy
│      ├── Pivot tables
│      ├── Merge
│      ├── Aggregation
│      └── Missing values
│
├── 6. Student Risk Analysis
│      ├── Low attendance
│      ├── Poor marks
│      ├── Low study hours
│      └── Risk score
│
├── 7. Performance Analytics
│      ├── Department
│      ├── Gender
│      ├── Subject
│      └── Semester
│
├── 8. Visualization
│      ├── Bar charts
│      ├── Histograms
│      ├── Scatter plots
│      ├── Box plots
│      └── Correlation heatmap
│
└── 9. Reporting
       ├── CSV
       ├── Excel
       └── Student report
```

# 28. Advanced Challenges for Students

To make this suitable as a **major MSc Data Science laboratory project**, give students these additional requirements:

1. Generate **1,000 synthetic student records using NumPy**.
2. Store them in Pandas DataFrames.
3. Introduce missing and inconsistent data.
4. Clean the dataset.
5. Build at least **8 Python classes**.
6. Demonstrate inheritance.
7. Demonstrate polymorphism.
8. Demonstrate encapsulation.
9. Use NumPy for statistical calculations.
10. Use Pandas for data manipulation.
11. Calculate student rankings.
12. Calculate subject-wise performance.
13. Calculate department-wise performance.
14. Calculate attendance statistics.
15. Identify academically weak students.
16. Develop a student **risk score**.

17. Find correlations between study hours, attendance and marks.
18. Create at least **5 visualizations**.
19. Export the final dataset to CSV and Excel.
20. Create a menu-driven interface.
21. Implement exception handling.
22. Validate user input.
23. Add logging.
24. Write reusable functions.
25. Organize the project into multiple Python files.

## Suggested final project structure

```
student_analytics/
│
├── main.py
├── student.py
├── academic.py
├── attendance.py
├── database.py
├── numpy_analysis.py
├── pandas_analysis.py
├── visualization.py
├── reports.py
├── utilities.py
│
├── data/
│   ├── students.csv
│   └── marks.csv
│
├── reports/
│   ├── student_report.csv
│   └── department_report.xlsx
│
└── charts/
    ├── performance.png
    ├── attendance.png
    ├── subjects.png
    └── correlation.png
```